

Langchain

LangChain is a powerful framework designed for developing applications that utilize large language models (LLMs). It provides modular components that allow developers to build complex applications by integrating various LLMs and tools seamlessly. The framework is particularly focused on enabling chat-based interactions, making it suitable for creating chatbots, virtual assistants, and other conversational agents.

Why Use LangChain?

- **Modularity:** LangChain's architecture allows developers to mix and match components, such as different LLMs, tools, and memory systems.
- **Integration with Multiple Providers:** LangChain supports a wide range of LLM providers, allowing users to choose the best model for their specific needs. This includes well-known models from OpenAI, Anthropic, and others.
- **Ease of Use:** With built-in templates and utilities, LangChain simplifies the process of setting up and managing LLMs, enabling rapid development.
- **Advanced Features:** The framework supports advanced functionalities like tool calling, structured outputs, and multi-modal capabilities (e.g., handling images or audio).

Chat Models in LangChain

Chat models in LangChain are designed to handle conversations through a sequence of messages rather than simple text inputs. This approach allows for more context-aware interactions. The framework provides an extensive list of chat model integrations from various providers:

Provider

Description

OpenAI	Well-known for its powerful language models.
Anthropic	Offers advanced conversational AI capabilities.
Ollama	Supports running open-source models locally.
Mistral	Focuses on efficient model deployment.
DeepSeek	Provides hosted chat models for various applications.

Developers can easily select a chat model by installing the appropriate package and initializing it within their application code.

Using Ollama with LangChain

Ollama is a specific provider that allows users to run open-source large language models locally. It simplifies the setup process by bundling model weights, configurations, and data into a single package defined by a Modelfile.

Key Features of Ollama

- *Local Execution:* Run models like Llama 2 on your local machine.
- *Multi-modal Support:* Ollama can handle various input types including text, images, and audio.
- *Easy Model Management:* Users can pull different models easily using commands such as `ollama pull <model-name>`.

Basic Setup Instructions

1. Installation:

To use the Ollama integration within LangChain, install the package:

```
pip install -qU langchain-ollama
```

2. Running a Model:

After installation, you can run a model using:

```
from langchain_ollama import ChatOllama

llm = ChatOllama(model="llama3")

response = llm.invoke("Hello!")

print(response.content)
```

3. Chaining Models:

You can create chains of prompts and models for more complex interactions:

```
from langchain_core.prompts import ChatPromptTemplate

prompt = ChatPromptTemplate.from_messages(

    ("system", "You are a helpful assistant."),

    ("human", "{input}")

)

chain = prompt | llm

result = chain.invoke(input="Translate 'Hello' to French.")

print(result.content)
```

Using LLaMa, DeepSeek, or Mistral with LangChain

LangChain supports integration with various advanced LLMs including LLaMa (Large Language Model Meta AI), DeepSeek, and Mistral. Each of these models has unique strengths:

- *LLaMa*: Known for its efficiency in generating coherent text across multiple domains.
- *DeepSeek*: Offers specialized capabilities for deep learning applications.
- *Mistral*: Focuses on high-performance inference suitable for real-time applications.

Example Usage

For instance, to use the LLaMa model with LangChain:

```
from langchain.chat_models import init_chat_model

model = init_chat_model("llama3", model_provider="ollama")

response = model.invoke("What is the capital of France?")

print(response.content)    # Output: "The capital of France is Paris."
```

Chat Models in Langchain

Chat Models are a core component of LangChain, designed to facilitate conversational interactions with language models. Unlike traditional Language Models (LLMs) that operate on simple text input and output, chat models use a sequence of messages as input and return messages as output, enabling more nuanced and context-aware conversations. LangChain does not serve its own ChatModels, but rather provides a standard interface for interacting with many different models.

Key Features and Concepts

1. *Message-Based Input/Output*: Chat Models accept a list of messages as input, allowing for a richer conversational experience.
2. *Roles*: Each message can be assigned a distinct role, such as user, AI, or system, clarifying the context of each message. Common roles include:
 - System: Provides the chat model with instructions on how to behave and gives additional context.
 - User: Represents input from a user interacting with the model.
 - Assistant: Represents a response from the model.
 - Tool: Used to pass the results of a tool invocation back to the model after external data or processing has been retrieved.
3. *Unified Message Format*: LangChain provides a unified message format that can be used across chat models, so users can work with different chat models without worrying about the specific details of the message format used by each model provider.
4. *Tool Calling*: Many chat models offer a native tool calling API, allowing LLMs to interact with external services, APIs, and databases.
5. *Structured Output*: Chat Models can respond in a structured format, such as JSON that matches a given schema.
6. *Multimodality*: The ability to work with data other than text; for example, images, audio, and video.

LangChain's Role

LangChain serves as an interface, allowing interaction with diverse language models from various providers rather than being a provider itself. It offers a consistent interface for working with chat models from different providers while offering additional features for monitoring, debugging, and optimizing the performance of applications that use LLMs.

Integrations

LangChain has integrations with many model providers¹². Some of the featured providers include:

- OpenAI
- Anthropic
- Groq
- MistralAI
- Google Vertex AI
- FireworksAI
- Ollama

Quick Start Example

The following example demonstrates how to interact with a ChatModel:

```
from langchain_ollama import ChatOllama

chat_model = ChatOllama(model="llama3.2")

messages = [
    {"role": "system", "content": "You are a helpful assistant."},
    {"role": "user", "content": "Hello! Can you explain how to use LangChain?"}
]

response = chat_model(messages)
print(response['content'])
```

How-To Guides

LangChain provides several how-to guides for more advanced usage of LLMs, including:

- How to cache ChatModel responses
- How to use ChatModels that support function calling
- How to stream responses from a ChatModel
- How to track token usage in a ChatModel call
- How to create a custom ChatModel

Distinctions Between LLMs and ChatModels

The primary distinction between LLMs and ChatModels lies in their input and output structure. LLMs operate on a text-based input and output, while ChatModels follow a message-based input and output system.

Additional Features

The newest generation of chat models offers additional capabilities⁷:

- *Tool calling*: Many popular chat models offer a native tool calling API.
- *Structured output*: A technique to make a chat model respond in a structured format, such as JSON that matches a given schema.
- *Multimodality*: The ability to work with data other than text; for example, images, audio, and video.

Vector Embedding & Semantic Search

Vector Embeddings

What They Are:

Vector embeddings are numerical representations of text (or other data) in a high-dimensional space. Each piece of text—be it a sentence, paragraph, or document—is transformed into a vector such that semantically similar texts are located close together in this space.

How They're Generated:

- Embedding Models:

You can use pre-trained models like OpenAI's text-embedding-ada-002, Hugging Face's Sentence Transformers (e.g., all-mpnet-base-v2), or multilingual models. These models map text into fixed-length vectors.

- Semantic Meaning:

The vectors capture the context, syntax, and semantics of the text. This means that even if two sentences use different words, if they express similar ideas, their vectors will be similar.

Semantic Search with Vector Stores

What Is Semantic Search:

Semantic search goes beyond traditional keyword-based matching by using vector similarity. When you issue a query, you convert the query into an embedding, then compare it with stored embeddings to find the most semantically similar pieces of text.

Vector Stores in LangChain:

- Purpose:

A vector store is essentially a database for these embeddings. It stores both the vector and any associated metadata or original text.

- Backends:

LangChain supports various vector stores (e.g., FAISS, Pinecone, Qdrant, Weaviate). These systems provide efficient nearest-neighbor search algorithms to quickly identify embeddings that are close to a given query vector.

- Usage in Retrieval:

In a Retrieval Augmented Generation (RAG) setup, the agent first queries the vector store to fetch the most relevant documents or text segments. This retrieved context is then supplied to a language model to generate more accurate and context-aware responses.

Document Loaders & Preprocessing

What are Document Loaders in LangChain?

Document Loaders are specialized components in the LangChain framework designed to facilitate the ingestion of various types of data into a standardized format that can be processed by language models (LLMs). They play a crucial role in transforming raw data from different sources into structured document objects, which consist of two main fields:

- *page_content*: This field contains the actual text content extracted from the source document.
- *metadata*: This field holds additional information about the document, such as its source, author, and other relevant attributes.

LangChain supports over 80 different document loaders that can handle a variety of formats, including PDFs, CSVs, Word documents, HTML files, JSON, and plain text. This versatility allows developers to create applications that can work with diverse data sources seamlessly.

Types of Document Loaders

1. *File Loaders*: These loaders are used to load documents from local files. Examples include:
 - PDFLoader: Loads PDF documents.
 - CSVLoader: Loads CSV files.
 - TextLoader: Loads plain text files.
2. *Web Loaders*: These loaders fetch data from online sources. Examples include:
 - WebBaseLoader: Loads content from web pages.
 - SeleniumURLLoader: Uses Selenium to scrape dynamic web content.
3. *Unstructured Data Loaders*: These loaders handle unstructured data from various sources like social media or APIs.
4. *Structured Data Loaders*: These loaders are designed for tabular data formats, enabling users to perform semantic searches on structured datasets.

- *PDF Files :*

```
from langchain.document_loaders import PyPDFLoader
```

```
pdf_loader = PyPDFLoader("path/to/your/document.pdf")
```

```
documents = pdf_loader.load()
```

- *CSV Files :*

```
from langchain.document_loaders import CSVLoader
```

```
csv_loader = CSVLoader(file_path="path/to/your/data.csv")
```

```
documents = csv_loader.load()
```

- *Word Documents (DOCX) :*

```
from langchain.document_loaders import DocxLoader
```

```
docx_loader = DocxLoader("path/to/your/document.docx")
```

```
documents = docx_loader.load()
```

- *JSON Files :*

```
from langchain.document_loaders import JSONLoader

json_loader = JSONLoader(file_path="path/to/your/data.json")

documents = json_loader.load()
```

Handling Large Documents Efficiently (Chunking Techniques)

When dealing with large documents, it's essential to break them down into smaller, manageable chunks. This process is known as chunking and is crucial for effective semantic search and retrieval tasks. Chunking helps maintain context and ensures that relevant information is grouped together.

Chunking Techniques

1. Fixed-size Chunking:

This method divides text into chunks of a specified size (e.g., number of tokens or characters) with optional overlap.

```
from langchain.text_splitter import CharacterTextSplitter

text_splitter = CharacterTextSplitter(chunk_size=256,
chunk_overlap=20)

chunks = text_splitter.create_documents([large_text])
```

2. Recursive Chunking: This approach iteratively splits the text based on specific criteria until the desired chunk size is achieved.

```
from langchain.text_splitter import  
RecursiveCharacterTextSplitter  
  
recursive_splitter =  
RecursiveCharacterTextSplitter(chunk_size=256,  
chunk_overlap=20)  
  
chunks = recursive_splitter.create_documents([large_text])
```

3. Specialized Chunking: For structured content like Markdown or LaTeX, specialized chunking methods preserve the original structure during the splitting process.

```
from langchain.text_splitter import MarkdownTextSplitter  
  
markdown_splitter = MarkdownTextSplitter(chunk_size=100,  
chunk_overlap=0)  
  
chunks =  
markdown_splitter.create_documents([markdown_text])
```

Integrating Document Loaders with Vector Embedding & RAG

Once documents are loaded and chunked, they can be transformed into vector embeddings for efficient semantic search. This integration is particularly powerful when combined with Retrieval-Augmented Generation (RAG), which enhances LLM capabilities by allowing them to retrieve relevant information dynamically based on user queries.

Example Implementation

Here's a complete example demonstrating how to load a PDF document, create embeddings for its content, store them in a vector database, and set up a RAG system for querying:

Step 1: Load the PDF document

```
from langchain.document_loaders import PyPDFLoader
```

```
pdf_loader = PyPDFLoader("path/to/document.pdf")
```

```
documents = pdf_loader.load()
```

Step 2: Chunk the document

```
from langchain.text_splitter import  
RecursiveCharacterTextSplitter
```

```
text_splitter = RecursiveCharacterTextSplitter(chunk_size=256,  
chunk_overlap=20)
```

```
chunks = text_splitter.create_documents([doc.page_content for  
doc in documents])
```

Step 3: Create embeddings

```
from langchain.embeddings import OpenAIEmbeddings
```

```
embeddings_model = OpenAIEmbeddings()
```

```
vectors = [embeddings_model.embed(chunk.page_content) for chunk  
in chunks]
```

Step 4: Store vectors in FAISS vector store

```
from langchain.vectorstores import FAISS
```

```
vector_store = FAISS.from_embeddings(vectors)
```

Step 5: Set up RAG with RetrievalQA

```
from langchain.chains import RetrievalQA
```

```
qa_chain = RetrievalQA.from_llm(llm=your_llm_model,  
retriever=vector_store.as_retriever())
```

Step 6: Ask questions about the document

```
query_result = qa_chain.invoke({"question": "What key points  
does this document cover?"})
```

```
print(query_result['answer'])
```